

BAB 3: Logika Pengambilan Keputusan dan Perulangan

1. Pendahuluan

Pada bab sebelumnya, Anda telah mempelajari cara menyimpan data (variabel) dan melakukan perhitungan (aritmatika). Namun, aplikasi yang hanya bisa menghitung belum cukup "cerdas". Aplikasi yang baik harus mampu **mengambil keputusan** dan **mengulang tugas** secara otomatis.

Analogi Persimpangan Jalan (Decision Making): Bayangkan Anda sedang berdiri di persimpangan jalan. Ada rambu lalu lintas yang memberi instruksi:

- **Jika** lampu hijau menyala, **maka** Anda boleh jalan.
- **Jika** lampu merah menyala, **maka** Anda harus berhenti.
- **Jika** lampu kuning, **maka** Anda bersiap-siap.

Inilah esensi dari **struktur percabangan** dalam pemrograman. Aplikasi Anda harus bisa "berpikir" dan mengambil keputusan berbeda berdasarkan kondisi yang dihadapi. Dalam konteks pendidikan, ini seperti sistem yang otomatis menentukan apakah siswa **Lulus** atau **Remedial** berdasarkan nilai yang dimasukkan.

Analogi Ban Berjalan (Looping): Sekarang bayangkan sebuah ban berjalan (conveyor belt) di pabrik. Ban ini terus bergerak mengulang pekerjaan yang sama sampai semua barang selesai diproses. Anda tidak perlu memerintahkan "angkat barang" sebanyak 1000 kali secara manual. Cukup perintah: "Ulangi sampai semua barang selesai."

Inilah esensi dari **struktur perulangan**. Dalam aplikasi pendidikan, perulangan berguna untuk:

- Menampilkan daftar nama siswa satu per satu.
- Memproses 50 jawaban ujian secara otomatis.
- Membuat animasi loading bar saat mengunggah tugas.

Mengapa Aplikasi Pendidikan Butuh Kecerdasan? Sebagai calon guru yang **Profesional**, Anda harus memastikan aplikasi yang Anda buat dapat:

1. **Memvalidasi Input:** Mencegah siswa memasukkan nilai di luar rentang (misal: nilai 150 untuk skala 0-100).
2. **Memberikan Feedback Otomatis:** Siswa langsung tahu apakah jawaban mereka benar atau salah.
3. **Menghemat Waktu Guru:** Perulangan memungkinkan pemrosesan data massal tanpa harus klik satu per satu.

Dengan menguasai logika percabangan dan perulangan, aplikasi Anda akan berubah dari "kalkulator pasif" menjadi "asisten cerdas" yang membantu proses pembelajaran.

2. Capaian Pembelajaran

Kode	Sasaran Pembelajaran (Sub-CPMK)
------	---------------------------------

Kode	Sasaran Pembelajaran (Sub-CPMK)
Sub-CPMK 4	Mahasiswa mampu menerapkan struktur kontrol percabangan (If-Then-Else, Select Case) untuk membangun alur keputusan aplikasi pendidikan secara tepat.
Sub-CPMK 5	Mahasiswa mampu menggunakan teknik perulangan (For...Next, Do...Loop) untuk optimasi pengolahan data massal secara efisien.

Indikator Pencapaian:

1. Mahasiswa dapat mengimplementasikan percabangan tunggal, ganda, dan jamak.
2. Mahasiswa dapat menentukan predikat nilai berdasarkan rentang skor menggunakan logika kondisional.
3. Mahasiswa dapat menggunakan perulangan For...Next dan Do...Loop sesuai konteks.
4. Mahasiswa mampu mengoptimasi kode dengan memilih struktur yang paling efisien.
5. Mahasiswa dapat mengubah properti objek secara dinamis melalui kode program.

3. Materi 1: Struktur Percabangan (Decision Making)

3.1. If...Then...Else (Kondisi Tunggal & Ganda)

Struktur **If...Then...Else** adalah bentuk percabangan paling dasar. Struktur ini memungkinkan program mengambil keputusan berdasarkan kondisi yang bernilai **True** (Benar) atau **False** (Salah).

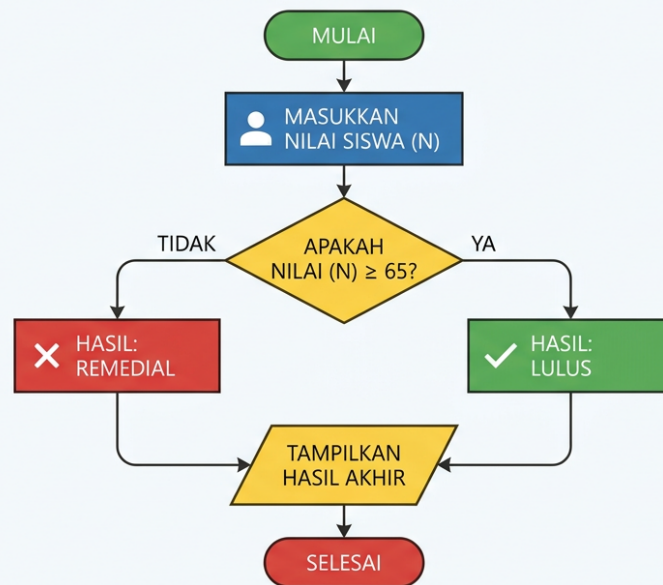
A. Kondisi Tunggal (If...Then) Digunakan ketika hanya ada satu kondisi yang perlu diperiksa.

```
If nilai >= 75 Then
    MessageBox.Show("Selamat, Anda Lulus!")
End If
```

B. Kondisi Ganda (If...Then...Else) Digunakan ketika ada dua kemungkinan hasil (Ya/Tidak, Lulus/Gagal).

```
If nilai >= 75 Then
    MessageBox.Show("Selamat, Anda Lulus!")
Else
    MessageBox.Show("Silakan mengikuti Remedial.")
End If
```

FLOWCHART IF-THEN-ELSE SEDERHANA: KEPUTUSAN KELULUSAN SISWA

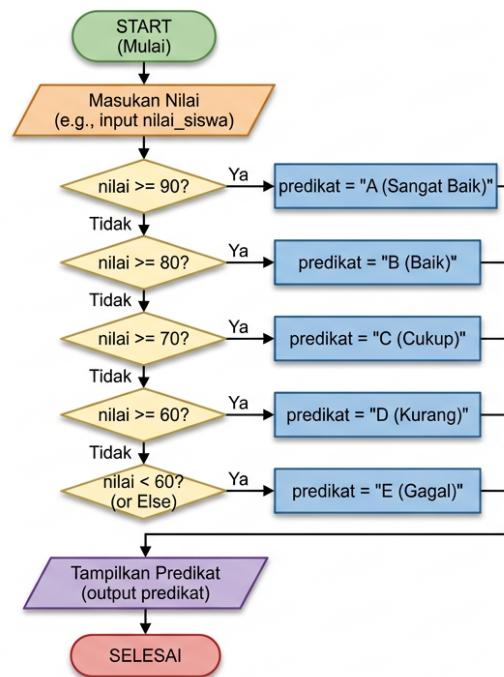


[GAMBAR 3.1: Flowchart If-Then-Else Sederhana]

C. Kondisi Bertingkat (If...Then...Elseif) Digunakan ketika ada lebih dari dua kemungkinan kondisi. Ini sangat berguna untuk menentukan predikat nilai.

```
If nilai >= 90 Then
    predikat = "A (Sangat Baik)"
ElseIf nilai >= 80 Then
    predikat = "B (Baik)"
ElseIf nilai >= 70 Then
    predikat = "C (Cukup)"
ElseIf nilai >= 60 Then
    predikat = "D (Kurang)"
Else
    predikat = "E (Gagal)"
End If
```

FLOWCHART IF-THEN-ELSEIF BERTINGKAT: MENENTUKAN PREDIKAT NILAI



[GAMBAR 3.2: Flowchart If-Then-Elseif Bertingkat]

3.2. Select Case (Kondisi Jamak/Banyak)

Ketika Anda memiliki banyak kondisi yang memeriksa **satu variabel yang sama**, struktur **Select Case** lebih rapi dan efisien daripada **If...ElseIf** yang bertumpuk.

Sintaks:

```

Select Case variabel
  Case nilai1
    ' Kode untuk kondisi 1
  Case nilai2
    ' Kode untuk kondisi 2
  Case Else
    ' Kode jika tidak ada yang cocok
End Select
  
```

Contoh: Menentukan Hari dari Angka

```

Select Case angkaHari
  Case 1
    lblHari.Text = "Senin"
  Case 2
    lblHari.Text = "Selasa"
  Case 3
    lblHari.Text = "Rabu"
  Case 4
    lblHari.Text = "Kamis"
  Case 5
    lblHari.Text = "Jumat"
  Case 6
    lblHari.Text = "Sabtu"
  Case 7
    lblHari.Text = "Minggu"
End Select
  
```

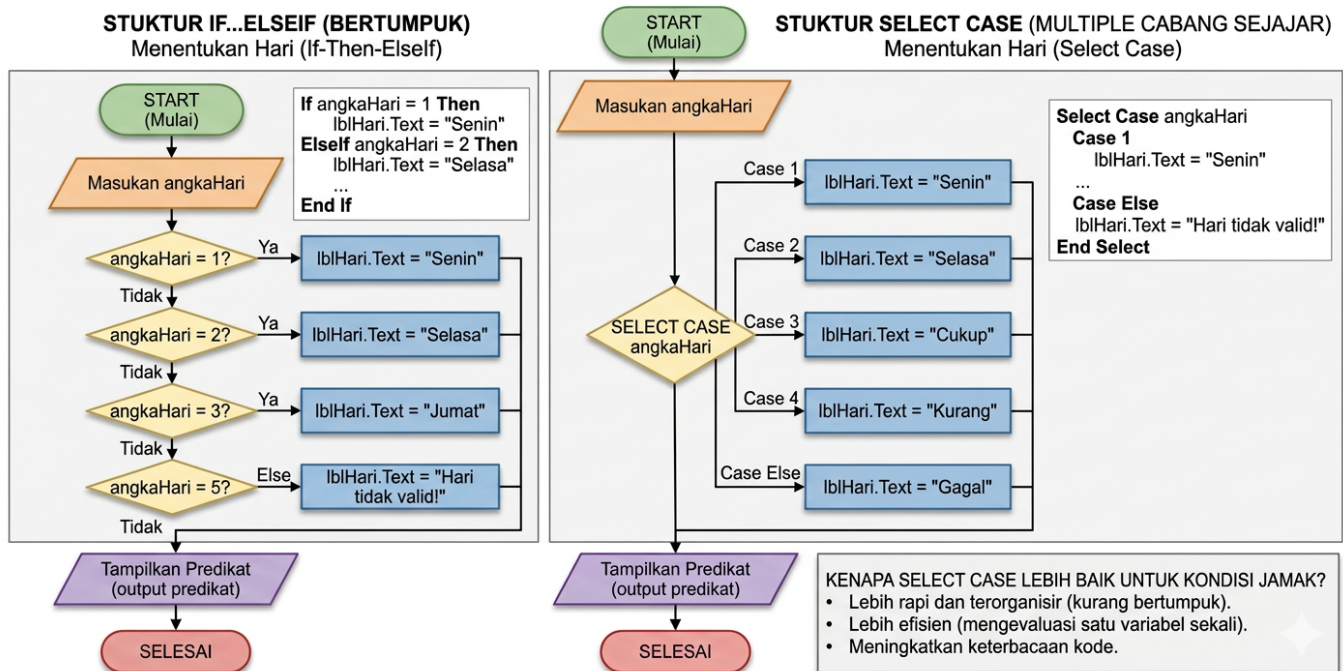


```

        lblHari.Text = "Jumat"
    Case 6
        lblHari.Text = "Sabtu"
    Case 7
        lblHari.Text = "Minggu"
    Case Else
        lblHari.Text = "Hari tidak valid!"
End Select

```

PERBANDINGAN STRUKTUR KONDISI: IF...ELSEIF vs SELECT CASE



[GAMBAR 3.3: Perbandingan If-Elseif vs Select Case]

3.3. Contoh Kasus: Menentukan Predikat Nilai

Berikut adalah implementasi lengkap untuk menentukan predikat nilai berdasarkan skor 0-100:

```
Public Class Form1
```

```
    Private Sub btnCekPredikat_Click(sender As Object, e As EventArgs) Handles
        btnCekPredikat.Click
```

```
        Dim nilai As Integer
        Dim predikat As String
        Dim warna As Color
```

```
        ' Validasi input
        If txtNilai.Text = "" Then
            MessageBox.Show("Harap masukkan nilai!", "Peringatan")
            Exit Sub
        End If
```

```
        nilai = CInt(txtNilai.Text)
```

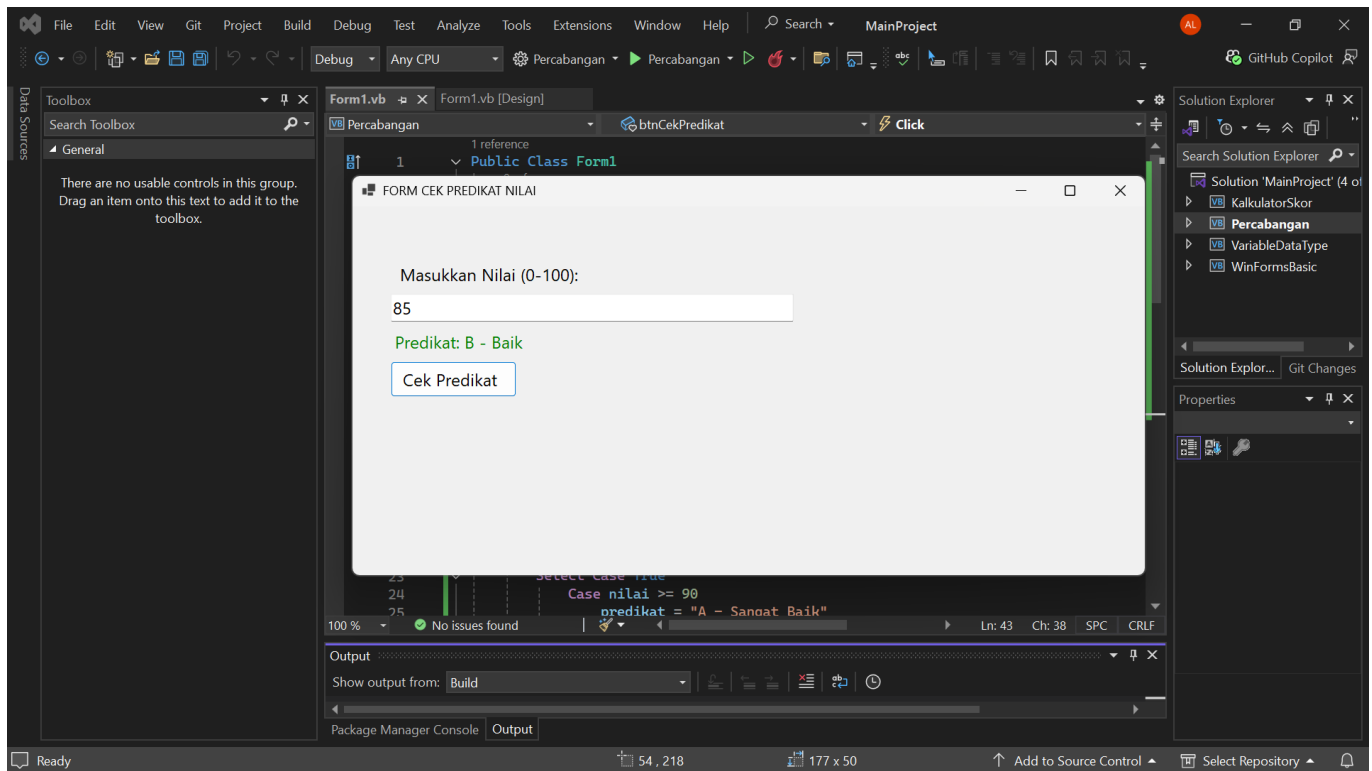
```
' Validasi rentang nilai
If nilai < 0 Or nilai > 100 Then
    MessageBox.Show("Nilai harus antara 0-100!", "Error")
Exit Sub
End If

' Menentukan predikat
Select Case True
    Case nilai >= 90
        predikat = "A - Sangat Baik"
        warna = Color.DarkGreen
    Case nilai >= 80
        predikat = "B - Baik"
        warna = Color.Green
    Case nilai >= 70
        predikat = "C - Cukup"
        warna = Color.Blue
    Case nilai >= 60
        predikat = "D - Kurang"
        warna = Color.Orange
    Case Else
        predikat = "E - Gagal"
        warna = Color.Red
End Select

' Menampilkan hasil
lblPredikat.Text = "Predikat: " & predikat
lblPredikat.ForeColor = warna

End Sub

End Class
```



[GAMBAR 3.4: Form Cek Predikat Nilai]

4. Materi 2: Struktur Perulangan (Looping)

4.1. For...Next (Perulangan Terukur/Pasti)

Gunakan **For...Next** ketika Anda **tahu pasti** berapa kali perulangan akan terjadi. Struktur ini menggunakan counter yang bertambah atau berkurang setiap iterasi.

Sintaks Dasar:

```
For counter = awal To akhir [Step langkah]
    ' Kode yang diulang
Next counter
```

Contoh 1: Menampilkan Angka 1-10

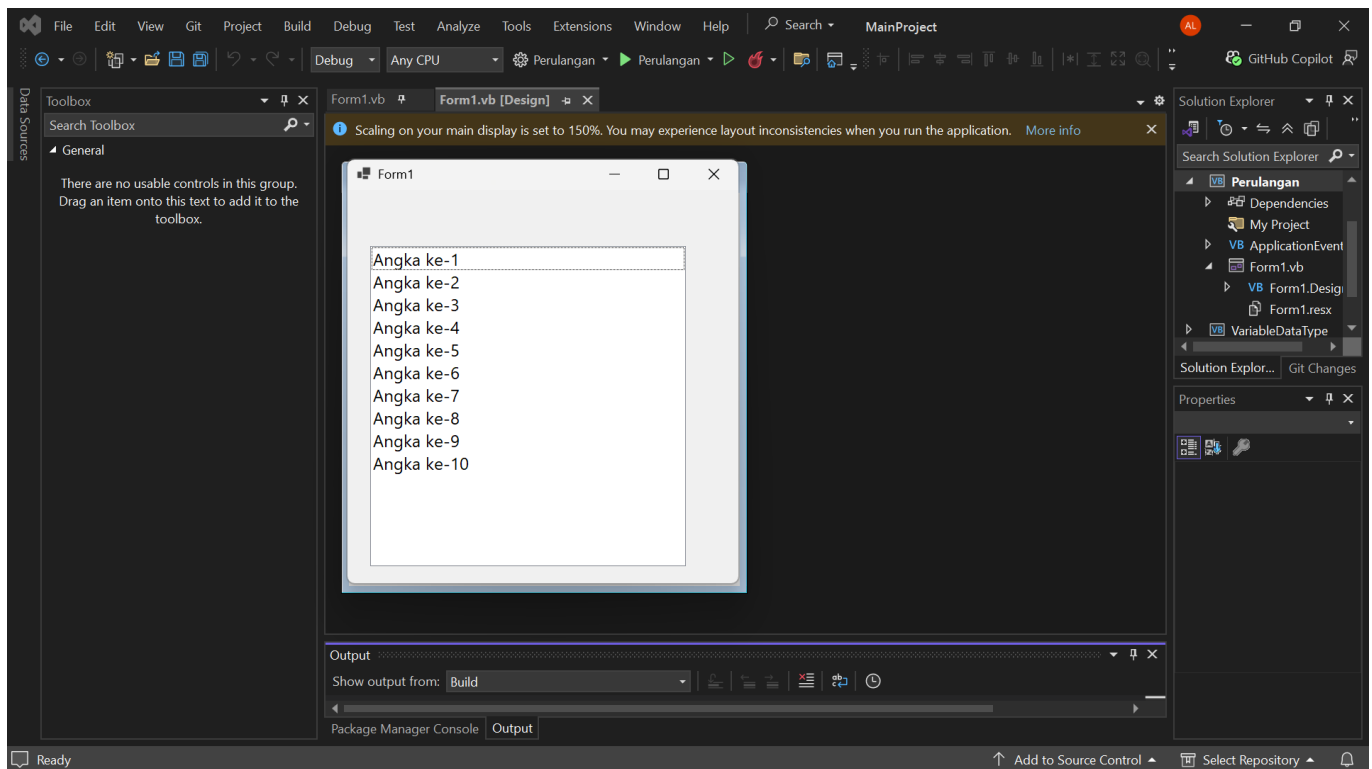
```
For i As Integer = 1 To 10
    ListBox1.Items.Add("Angka ke-" & i)
Next i
```

Contoh 2: Menampilkan Angka Genap 1-100

```
For i As Integer = 2 To 100 Step 2
    ListBox1.Items.Add(i)
Next i
```

Contoh 3: Perulangan Mundur (Countdown)

```
For i As Integer = 10 To 1 Step -1
    lblCountdown.Text = i.ToString()
    ' Delay 1 detik (perlu threading untuk implementasi real)
Next i
MessageBox.Show("Waktu Habis!")
```



[GAMBAR 3.5: Hasil Perulangan For-Next di ListBox]

4.2. Do...Loop (Perulangan Berdasarkan Kondisi)

Gunakan **Do...Loop** ketika Anda **tidak tahu pasti** berapa kali perulangan akan terjadi, tetapi tahu kondisi berhentinya.

A. Do While...Loop (Cek Kondisi di Awal)

```
Do While kondisi
    ' Kode yang diulang
Loop
```

Contoh:

```
Dim i As Integer = 1
Do While i <= 10
```

```

        ListBox1.Items.Add(i)
        i = i + 1
    Loop

```

B. Do...Loop While (Cek Kondisi di Akhir)

```

Do
    ' Kode yang diulang
Loop While kondisi

```

Perbedaan Penting:

- **Do While**: Kondisi dicek **sebelum** eksekusi. Bisa saja tidak pernah dijalankan jika kondisi awal False.
- **Do...Loop While**: Kondisi dicek **setelah** eksekusi. Minimal dijalankan **sekali**.



[GAMBAR 3.6: Flowchart Do-While vs Do-Loop-While]

4.3. Contoh Kasus: Simulasi Progress Bar

Berikut adalah contoh penggunaan perulangan untuk membuat animasi progress bar sederhana:

```

Private Sub btnUpload_Click(sender As Object, e As EventArgs) Handles
    btnUpload.Click

        ' Disable tombol saat proses
        btnUpload.Enabled = False

        ' Perulangan untuk simulasi upload
        For i As Integer = 0 To 100 Step 5
            ProgressBar1.Value = i
            lblStatus.Text = "Mengunggah..." & i & "%"
        Next i
    End Sub

```

```

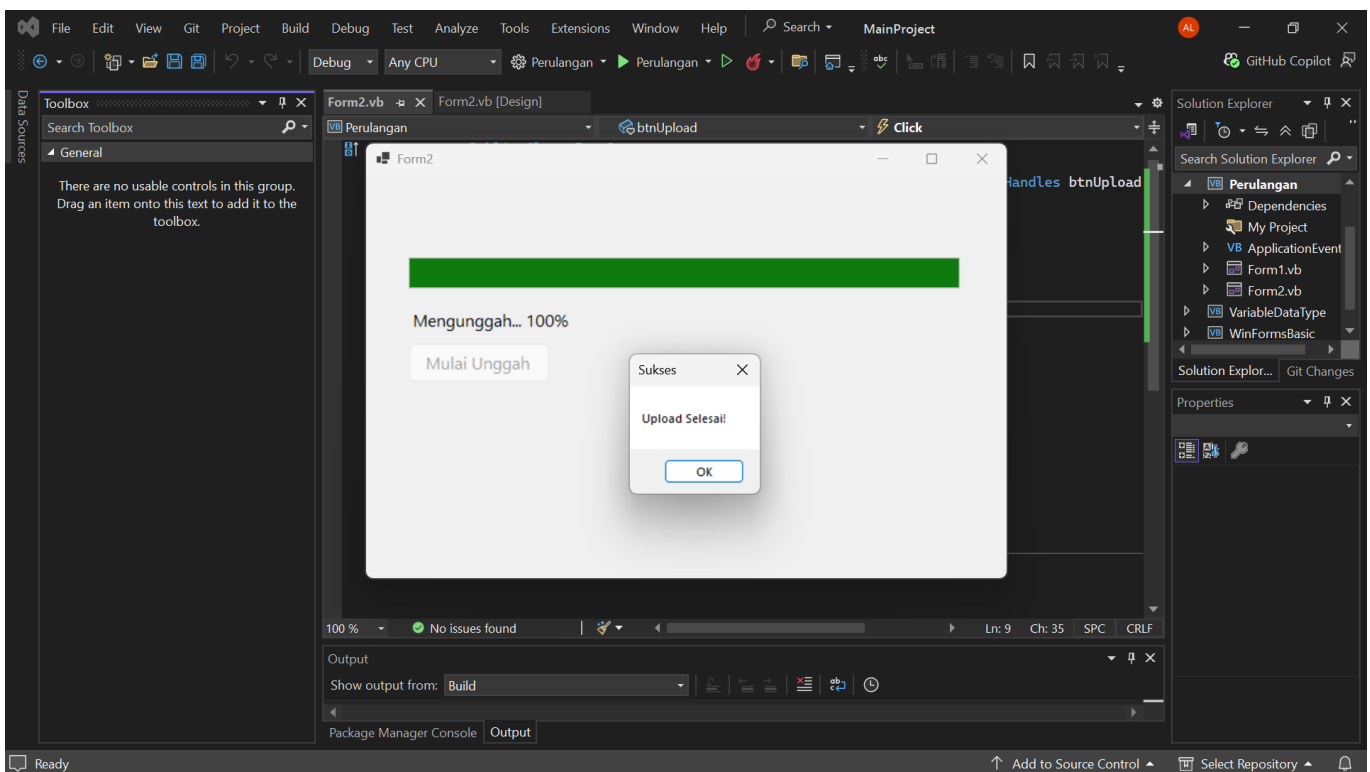
' Delay singkat untuk efek animasi
System.Threading.Thread.Sleep(100)

' Refresh UI
Application.DoEvents()
Next i

MessageBox.Show("Upload Selesai!", "Sukses")
btnUpload.Enabled = True

End Sub

```



[GAMBAR 3.7: Simulasi Progress Bar Upload]

5. Materi 3: Optimasi Logika

5.1. Nested If (If di dalam If)

Nested If adalah struktur percabangan di dalam percabangan lain. Ini digunakan ketika ada kondisi bertingkat yang harus dipenuhi.

Contoh: Validasi Login Sederhana

```

If txtUsername.Text = "admin" Then
    If txtPassword.Text = "12345" Then
        MessageBox.Show("Login Berhasil!")
        frmDashboard.Show()
    Else

```

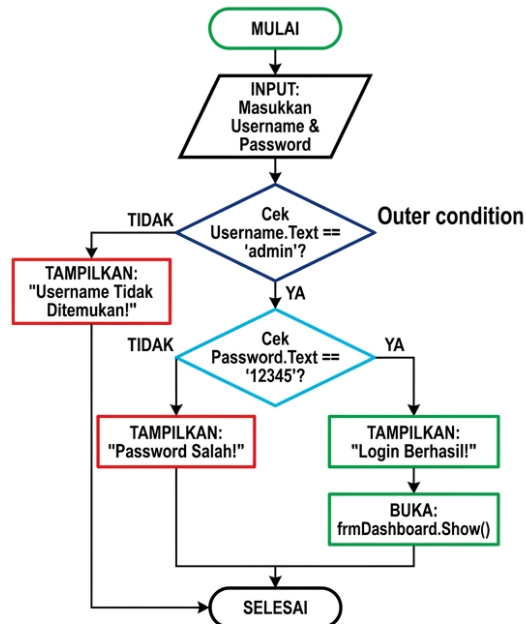


```

        MessageBox.Show("Password Salah!")
    End If
Else
    MessageBox.Show("Username Tidak Ditemukan!")
End If

```

DIAGRAM FLOWCHART: VALIDASI LOGIN (Percabangan Bersarang)



[GAMBAR 3.8: Diagram Nested If]

Kapan Menggunakan Nested If?

- Ketika kondisi kedua **hanya relevan** jika kondisi pertama terpenuhi.
- Contoh: Hanya cek password jika username sudah benar.

Risiko: Terlalu banyak nested if membuat kode sulit dibaca (disebut "Spaghetti Code"). Usahakan maksimal 3 tingkat kedalaman.

5.2. Nested Loop (Perulangan Bersarang)

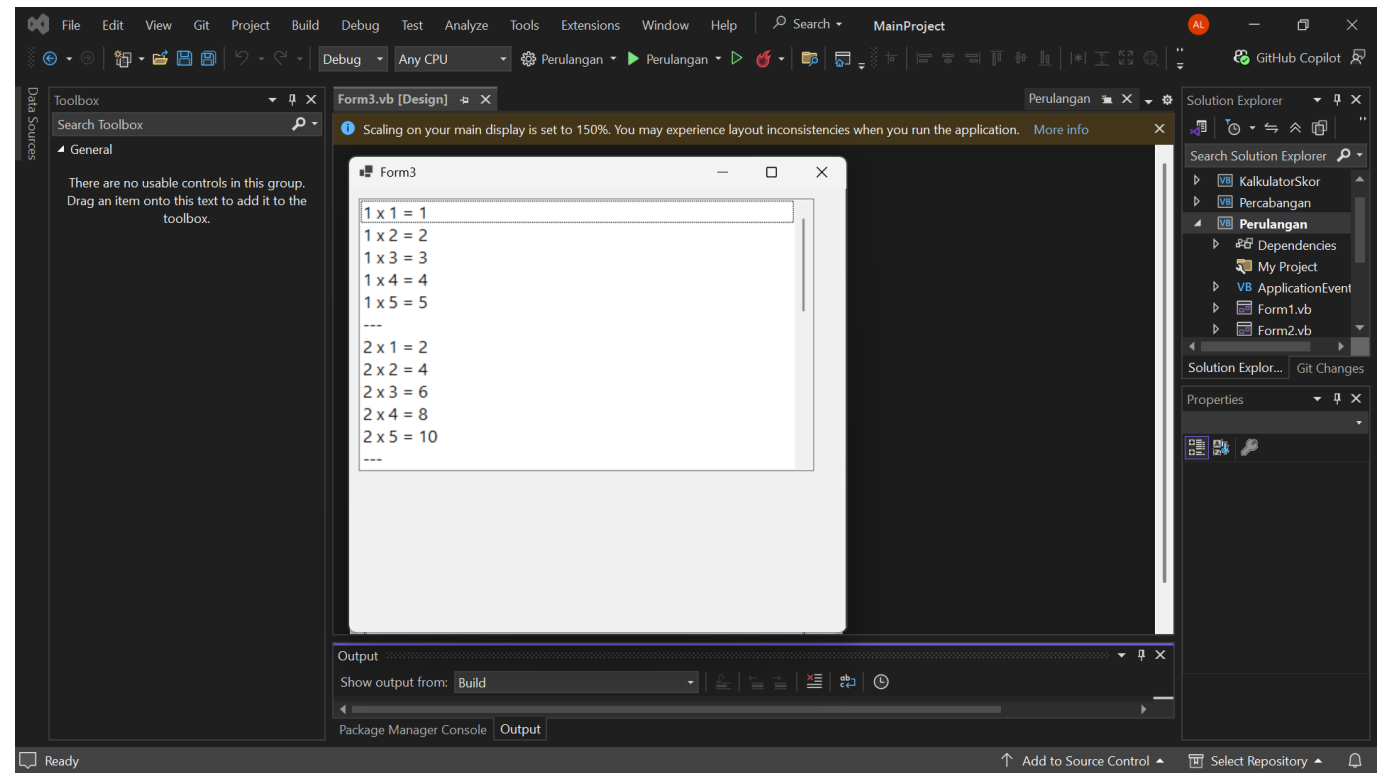
Nested Loop adalah perulangan di dalam perulangan lain. Ini berguna untuk membuat pola, tabel, atau matriks.

Contoh: Tabel Perkalian

```

For i As Integer = 1 To 5
    For j As Integer = 1 To 5
        Dim hasil As Integer = i * j
        ListBox1.Items.Add(i & " x " & j & " = " & hasil)
    Next j
    ListBox1.Items.Add("---") ' Pemisah
Next i

```



[GAMBAR 3.9: Hasil Nested Loop Tabel Perkalian]

5.3. Kapan Memilih Select Case daripada If Bertumpuk?

Kriteria	Gunakan If-Elseif	Gunakan Select Case
Kondisi berbeda variabel	☒ Ya	☒ Tidak
Satu variabel, banyak nilai	☒ Kurang efisien	☒ Lebih rapi
Kondisi kompleks (AND/OR)	☒ Ya	☒ Terbatas
Keterbacaan kode	Menurun jika >5 kondisi	Tetap rapi

Rekomendasi: Jika Anda memeriksa **satu variabel** dengan **lebih dari 3 kondisi**, gunakan **Select Case**. Ini membuat kode lebih mudah dibaca dan dipelihara—cerminan sikap **Amanah** dalam penulisan kode profesional.

6. Implementasi Kode: Cek Kelulusan Otomatis

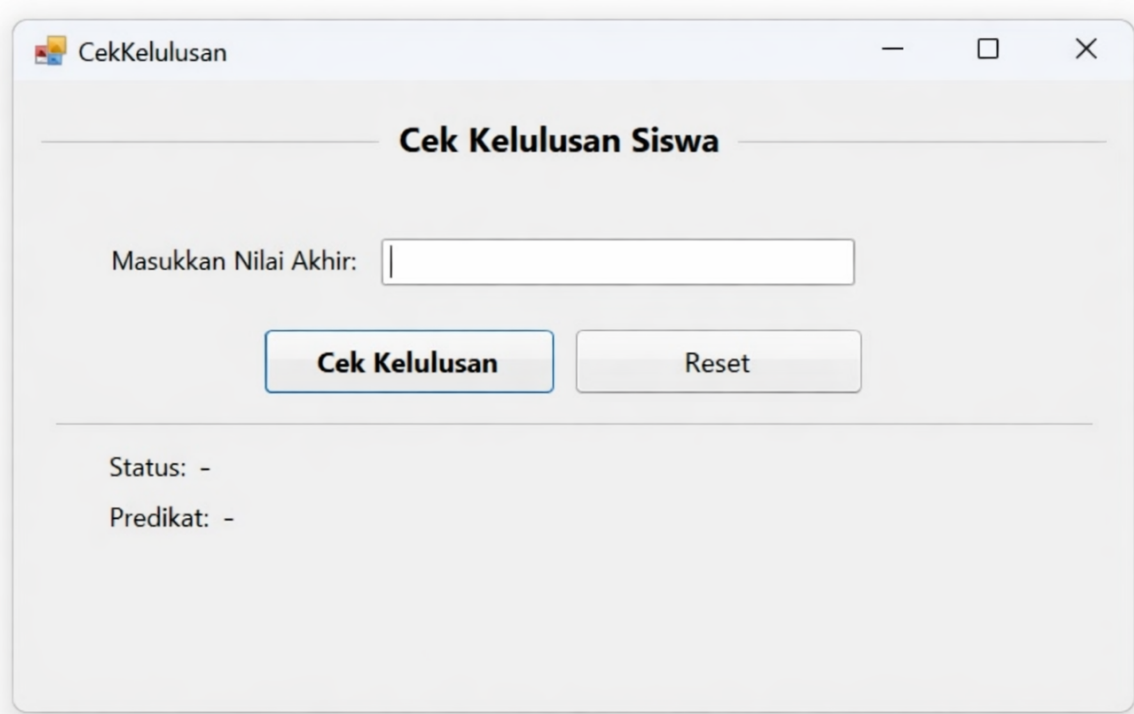
Mari kita terapkan semua konsep di atas dengan membuat aplikasi "**Cek Kelulusan Otomatis**". Aplikasi ini akan memvalidasi input, menentukan status kelulusan, dan mengubah warna label secara dinamis.

Langkah 1: Desain Form

Buat proyek baru bernama **CekKelulusan**. Desain form dengan komponen berikut:

Komponen	Nama (Name)	Text/Properties	Keterangan
Label	lblJudul	"Cek Kelulusan Siswa"	Font Bold, Size 14

Komponen	Nama (Name)	Text/Properties	Keterangan
Label	lblNilai	"Masukkan Nilai Akhir:"	-
TextBox	txtNilai	-	Input nilai 0-100
Button	btnCek	"Cek Kelulusan"	Tombol proses
Button	btnReset	"Reset"	Tombol kosongkan
Label	lblStatus	"Status: -"	Output status
Label	lblPredikat	"Predikat: -"	Output predikat



[GAMBAR 3.10: Desain Form Cek Kelulusan]

Langkah 2: Menulis Kode Program

Klik dua kali tombol `btnCek` dan masukkan kode berikut:

```
Public Class Form1
    ' Konstanta untuk KKM
    Const KKM As Integer = 75

    Private Sub btnCek_Click(sender As Object, e As EventArgs) Handles
        btnCek.Click
            ' Deklarasi variabel
```

```
Dim nilai As Integer
Dim status As String
Dim predikat As String
Dim warnaStatus As Color

' === VALIDASI INPUT ===
' Cek apakah TextBox kosong
If txtNilai.Text = "" Then
    MessageBox.Show("Harap masukkan nilai terlebih dahulu!", "Peringatan",
        MessageBoxButtons.OK, MessageBoxIcon.Warning)
    txtNilai.Focus()
    Exit Sub
End If

' Cek apakah input adalah angka
If Not IsNumeric(txtNilai.Text) Then
    MessageBox.Show("Input harus berupa angka!", "Error",
        MessageBoxButtons.OK, MessageBoxIcon.Error)
    txtNilai.Clear()
    txtNilai.Focus()
    Exit Sub
End If

' Konversi dan validasi rentang
nilai = CInt(txtNilai.Text)

If nilai < 0 Or nilai > 100 Then
    MessageBox.Show("Nilai harus antara 0 sampai 100!", "Error",
        MessageBoxButtons.OK, MessageBoxIcon.Error)
    txtNilai.Clear()
    txtNilai.Focus()
    Exit Sub
End If

' === LOGIKA PERCABANGAN ===
' Menentukan status kelulusan
If nilai >= KKM Then
    status = "LULUS"
    warnaStatus = Color.Green
Else
    status = "REMEDIAL"
    warnaStatus = Color.Red
End If

' Menentukan predikat
Select Case True
    Case nilai >= 90
        predikat = "A (Sangat Baik)"
    Case nilai >= 80
        predikat = "B (Baik)"
    Case nilai >= 70
        predikat = "C (Cukup)"
    Case nilai >= 60
        predikat = "D (Kurang)"
```

```

        Case Else
            predikat = "E (Gagal)"
        End Select

' === TAMPILKAN HASIL ===
lblStatus.Text = "Status: " & status
lblStatus.ForeColor = warnaStatus
lblStatus.Font = New Font(lblStatus.Font, FontStyle.Bold)

lblPredikat.Text = "Predikat: " & predikat

MessageBox.Show("Status: " & status & vbCrLf & "Predikat: " & predikat,
    "Hasil Kelulusan", MessageBoxButtons.OK,
    MessageBoxIcon.Information)

End Sub

Private Sub btnReset_Click(sender As Object, e As EventArgs) Handles
    btnReset.Click
        ' Reset semua input dan output
        txtNilai.Clear()
        lblStatus.Text = "Status: -"
        lblStatus.ForeColor = Color.Black
        lblPredikat.Text = "Predikat: -"
        txtNilai.Focus()
    End Sub

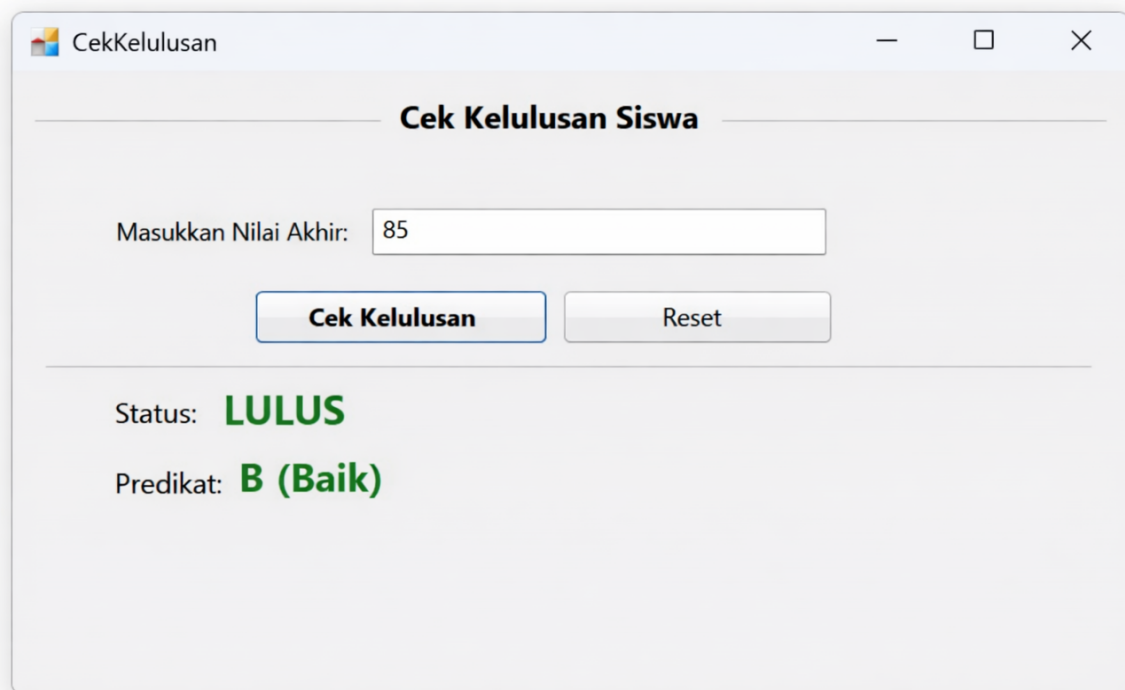
End Class

```

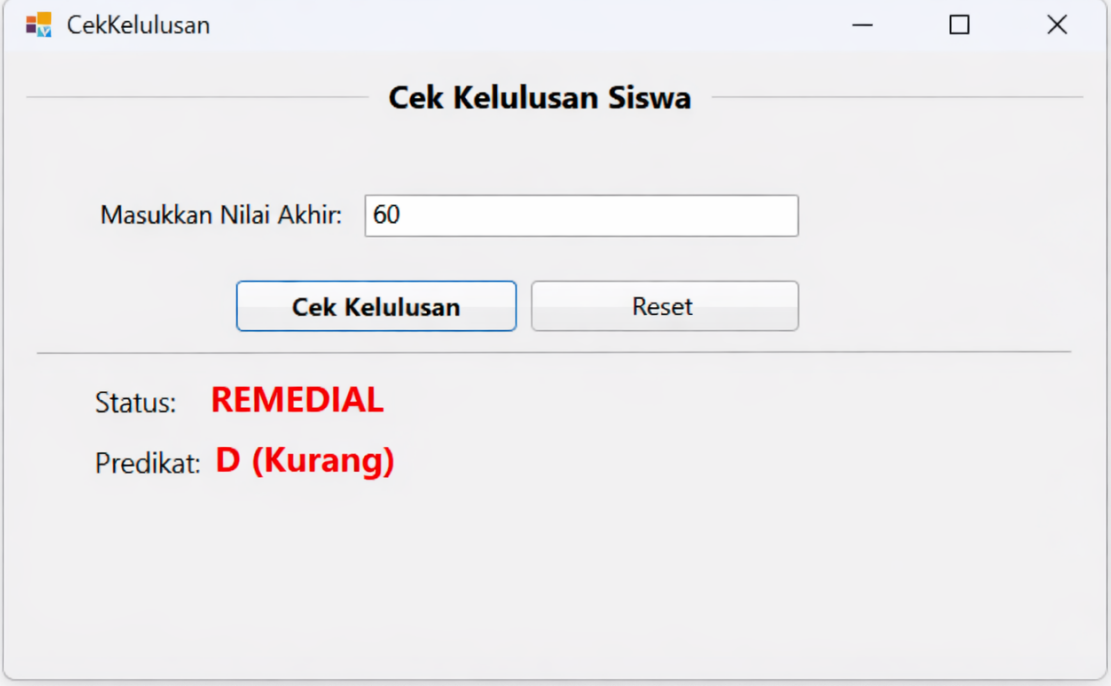
Penjelasan Baris per Baris

No	Kode	Penjelasan
1-4	<code>Const KKM As Integer = 75</code>	Deklarasi konstanta KKM untuk standar kelulusan.
5	<code>Private Sub btnCek_Click...</code>	Event handler saat tombol Cek diklik.
6-10	<code>Dim ...</code>	Deklarasi variabel untuk status, predikat, dan warna.
11-17	<code>If txtNilai.Text = "" Then...</code>	Validasi 1: Cek apakah input kosong.
18-24	<code>If Not IsNumeric(...) Then...</code>	Validasi 2: Cek apakah input adalah angka.
25-32	<code>If nilai < 0 Or nilai > 100 Then...</code>	Validasi 3: Cek rentang nilai 0-100.
33-39	<code>If nilai >= KKM Then...</code>	Percabangan untuk menentukan Lulus/Remedial.
40-50	<code>Select Case True...</code>	Percabangan jamak untuk menentukan predikat A-E.

No	Kode	Penjelasan
51- 54	<code>lblStatus.ForeColor = warnaStatus</code>	Perubahan properti dinamis - warna label berubah sesuai status.
55- 57	<code>MessageBox.Show(...)</code>	Menampilkan popup hasil kelulusan.
58- 64	<code>btnReset_Click...</code>	Event handler untuk tombol Reset.

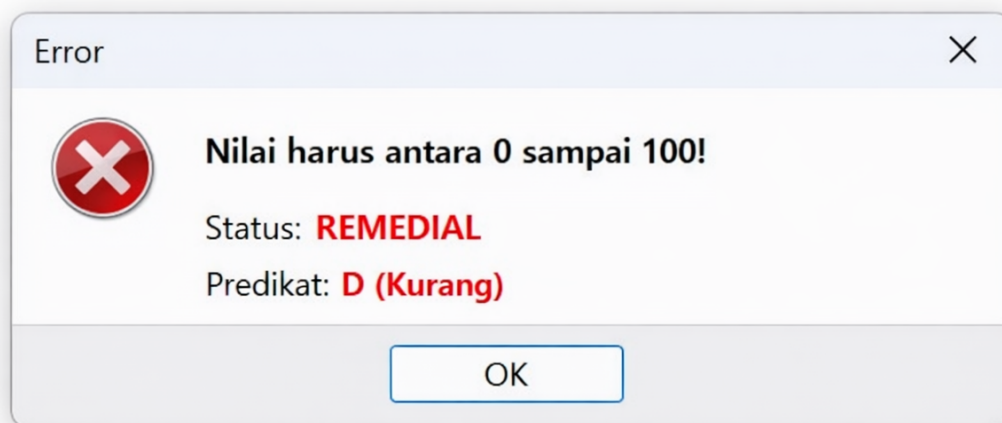


[GAMBAR 3.12: Hasil Running - Lulus (Hijau)]



The screenshot shows a Windows application window titled "CekKelulusan". Inside the window, the title "Cek Kelulusan Siswa" is centered at the top. Below the title, there is a label "Masukkan Nilai Akhir:" followed by a text input field containing the number "60". Underneath the input field are two buttons: "Cek Kelulusan" and "Reset". Below a horizontal separator line, the application displays the results: "Status: **REMEDIAL**" and "Predikat: **D (Kurang)**".

[GAMBAR 3.13: Hasil Running - Remedial (Merah)]



[GAMBAR 3.14: MessageBox Error Input Tidak Valid]

7. Latihan Praktikum Mandiri

Kasus 1: Error Prevention pada Kalkulator

Deskripsi: Modifikasi aplikasi Kalkulator dari Bab 2 agar tidak dapat membagi angka dengan nol. Ini adalah bentuk **Error Prevention** yang penting dalam pengembangan aplikasi.

Spesifikasi:

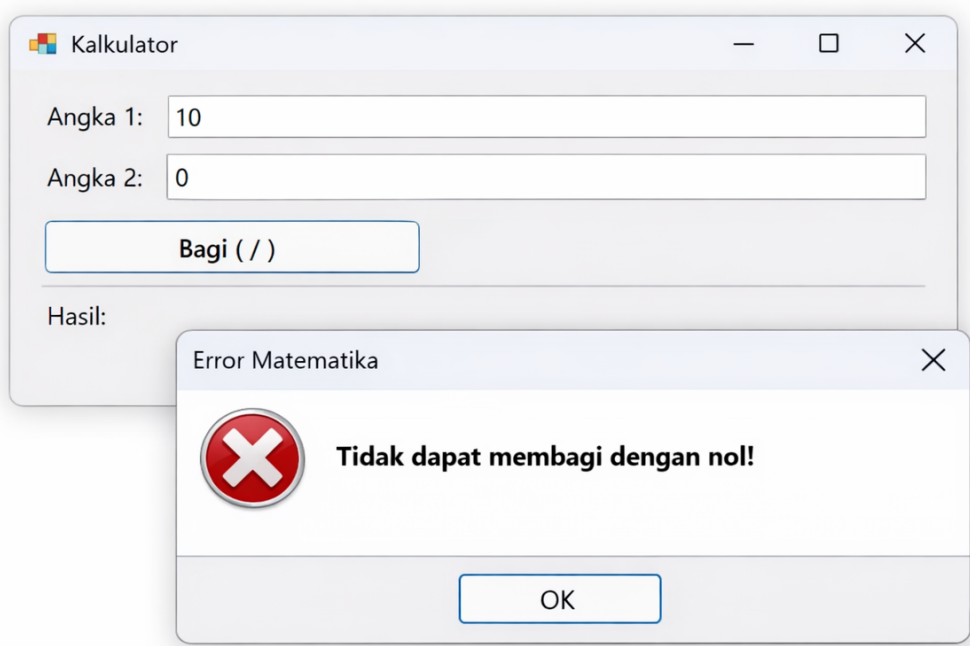
1. Buka kembali proyek **KalkulatorSkor** dari Bab 2.
2. Tambahkan validasi pada tombol Hitung:
 - Jika penyebut (divisor) = 0, tampilkan MessageBox error.
 - Jangan lakukan perhitungan jika input tidak valid.
3. Gunakan **Try...Catch** untuk menangani error tak terduga.

Contoh Kode Validasi:

```
Private Sub btnBagi_Click(...) Handles btnBagi.Click
    Dim angka1 As Double = CDb1(txtAngka1.Text)
    Dim angka2 As Double = CDb1(txtAngka2.Text)

    If angka2 = 0 Then
        MessageBox.Show("Tidak dapat membagi dengan nol!", "Error Matematika")
        Exit Sub
    End If

    Dim hasil As Double = angka1 / angka2
    lblHasil.Text = hasil.ToString()
End Sub
```



[GAMBAR 3.15: Form Kalkulator dengan Validasi Pembagian Nol]

Kasus 2: Tabel Perkalian Otomatis dengan ListBox

Deskripsi: Buat aplikasi "Tabel Perkalian Otomatis" yang menggunakan perulangan untuk menampilkan hasil perkalian 1-10 di ListBox.

Spesifikasi:

Komponen	Nama (Name)	Text/Properties
Form	Form1	Text = "Tabel Perkalian"
Label	lblAngka	"Masukkan Angka:"
TextBox	txtAngka	-
Button	btnTampilkan	"Tampilkan Tabel"
Button	btnClear	"Clear"
ListBox	lstHasil	-

Kode yang Diharapkan:

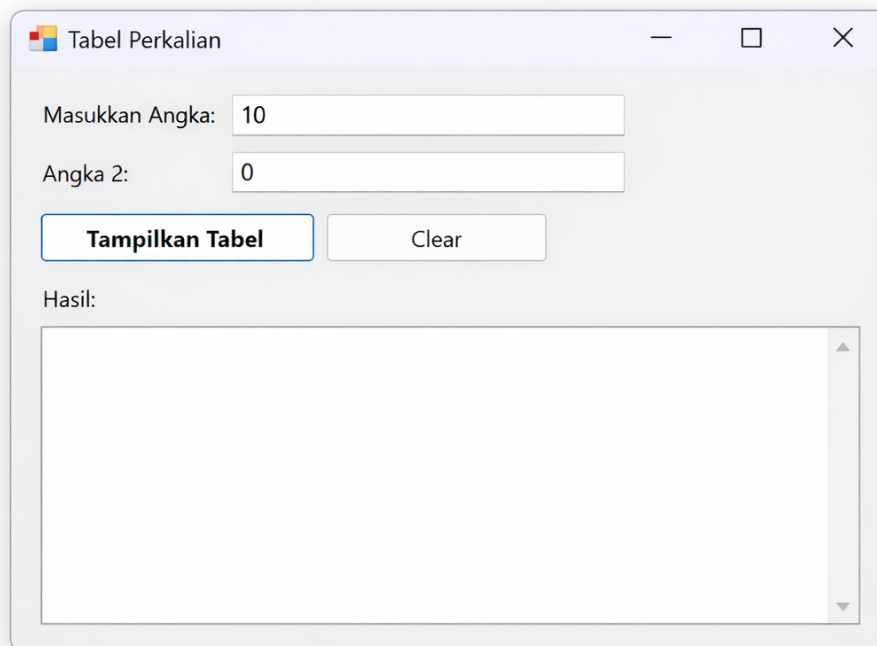
```
Private Sub btnTampilkan_Click(...) Handles btnTampilkan.Click
    ' Clear ListBox terlebih dahulu
    lstHasil.Items.Clear()
```

```
Dim angka As Integer = CInt(txtAngka.Text)

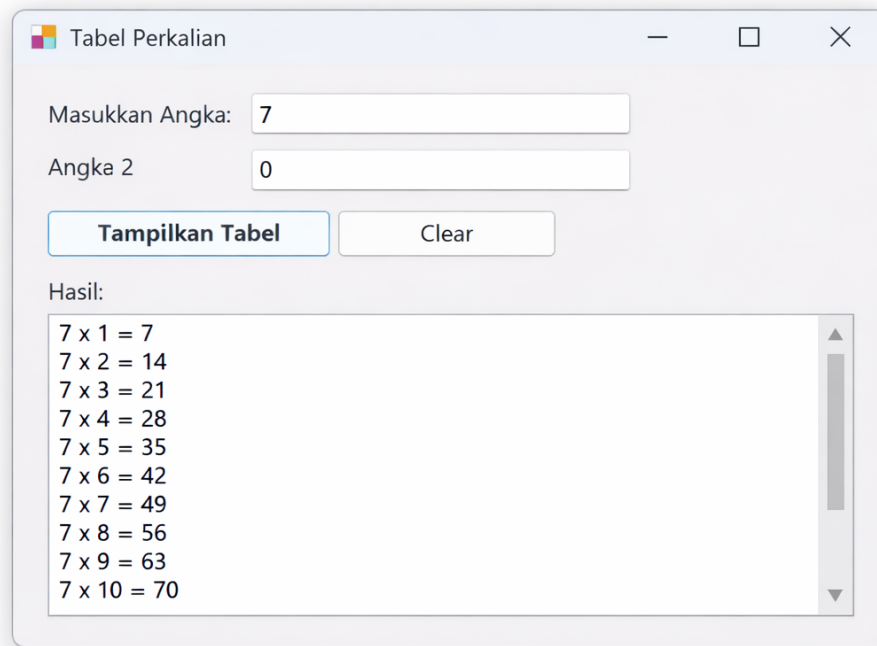
' Validasi
If angka < 1 Or angka > 100 Then
    MessageBox.Show("Masukkan angka 1-100!")
    Exit Sub
End If

' Perulangan untuk tabel perkalian
For i As Integer = 1 To 10
    Dim hasil As Integer = angka * i
    lstHasil.Items.Add(angka & " x " & i & " = " & hasil)
Next i
End Sub

Private Sub btnClear_Click(...) Handles btnClear.Click
    txtAngka.Clear()
    lstHasil.Items.Clear()
    txtAngka.Focus()
End Sub
```



[GAMBAR 3.16: Form Tabel Perkalian]



[GAMBAR 3.17: Hasil Running Tabel Perkalian di ListBox]

8. Evaluasi & Rangkuman

A. Soal Analisis Alur Program (Tracing Kode)

Soal 1: Perhatikan kode berikut:

```
Dim nilai As Integer = 85
Dim status As String

If nilai >= 90 Then
    status = "A"
ElseIf nilai >= 80 Then
    status = "B"
ElseIf nilai >= 70 Then
    status = "C"
Else
    status = "D"
End If
```

Pertanyaan: Apa nilai variabel `status` setelah kode dieksekusi? Jelaskan alurnya!

Jawaban:

Soal 2: Perhatikan kode perulangan berikut:

```
Dim total As Integer = 0
For i As Integer = 1 To 5
    total = total + i
Next i
```

Pertanyaan: Berapa nilai akhir variabel `total`? Tunjukkan proses iterasi!

Jawaban:

Soal 3: Perhatikan kode berikut:

```
Dim x As Integer = 10
Do While x > 5
    MessageBox.Show(x.ToString())
    x = x - 2
Loop
```

Pertanyaan: Berapa kali MessageBox akan muncul? Sebutkan nilai yang ditampilkan!

Jawaban:

B. Poin Kunci: Perbedaan For-Next vs Do-Loop

Aspek	For...Next	Do...Loop
Jumlah Iterasi	Diketahui sebelumnya	Tidak selalu diketahui
Counter	Otomatis (dengan Step)	Manual (harus di-update)
Kondisi	Based on range (To)	Based on True/False
Minimal Eksekusi	Minimal 1 kali (jika valid range)	0 kali (Do While) atau 1 kali (Loop While)
Use Case	Looping array, tabel, counting	Waiting condition, input validation

C. Rangkuman Bab 3

No	Poin Kunci	Deskripsi Singkat
1	If-Then-Else	Percabangan untuk 2 kondisi (True/False).
2	If-Elseif	Percabangan bertingkat untuk banyak kondisi.
3	Select Case	Lebih rapi untuk 1 variabel dengan banyak nilai.
4	For-Next	Perulangan dengan jumlah iterasi pasti.

No	Poin Kunci	Deskripsi Singkat
5	Do-Loop	Perulangan berdasarkan kondisi True/False.
6	Nested If	If di dalam If untuk validasi bertingkat.
7	Nested Loop	Loop di dalam Loop untuk pola/tabel.
8	Validasi Input	Wajib dilakukan sebelum memproses data.
9	Properti Dinamis	Ubah warna/font label via kode untuk feedback visual.
10	Error Prevention	Cegah error seperti pembagian dengan nol.

Penutup

Selamat! Anda telah menyelesaikan Bab 3. Sekarang aplikasi Anda memiliki "otak" untuk mengambil keputusan dan "otot" untuk mengulang tugas secara otomatis. Ini adalah lompatan besar dari aplikasi pasif menjadi aplikasi interaktif yang cerdas.